Software Engineering Lab

Lab 33

Read Replicas & Replication Lag

Mitigasi Stale Read & Menjaga Konsistensi Read-Your-Own-Writes

Lukman / lukman-ss

Anomali Stale Read pada Read Replica

User menulis data baru, namun query berikutnya membaca data lama atau kosong

GEJALA ANOMALI

User melakukan write profil atau checkout pesanan ke primary node
Request read berikutnya dialihkan round-robin ke replica
Replica belum menerapkan WAL entry terbaru, menghasilkan data basi atau 404

AKAR TEKNIS

Replikasi asinkron default pada PostgreSQL, MySQL, Aurora, dan MongoDB
Ada jeda propagasi jaringan dan waktu commit WAL antara master dan replica
Replica menjamin eventual consistency, bukan linearizability real-time

DAMPAK PENGGUNA

User mengira transaksi gagal lalu mengklik tombol submit berulang kali
Saldo rekening atau item keranjang belanja terlihat hilang sesaat
Inkonsistensi session yang merusak kepercayaan pengguna terhadap sistem

Ketika arsitektur basis data memisahkan beban tulis ke primary dan beban baca ke beberapa replika sekunder, replikasi asinkron menimbulkan celah propagasi. Pengguna yang baru saja memperbarui data dapat langsung diarahkan ke replika yang belum menerapkan perubahan tersebut. Akibatnya, sistem menyajikan data lama atau bahkan melaporkan data tidak ditemukan, memicu kebingungan operasional dan duplikasi transaksi.

Replication lag is the distance between primary commit and replica apply.

Anatomi Aliran Asynchronous Replication

Write commit langsung di primary sementara replica mengejar secara background

ALIRAN WAL ENTRY

1. Primary menerima write, catat data, naikkan LSN monotonik
2. Primary kembalikan status commit sukses ke client tanpa menunggu replica
3. WAL entry disalurkan via stream jaringan ke antrean worker replica
4. Worker replica memproses dan menerapkan entry ke memory/disk lokal

PERSAMAAN LAG

Replication Lag = LSN_primary - AppliedLSN_replica
Lag Waktu (Δt) = Waktu Terima Primary - Waktu Apply Replica
Kapasitas antrean WAL dan beban I/O lokal menentukan besaran lag

TRADE-OFF UTAMA

Keuntungan: Latensi write sangat rendah dan throughput primary maksimal
Konsekuensi: Jendela waktu staleness tidak terhindarkan bagi replica read

Dalam mode replikasi asinkron, primary memproses transaksi, mencatat entri Write-Ahead Log dengan nomor urut monotonik LSN, lalu seketika mengembalikan respons sukses ke aplikasi. Salinan log dikirimkan ke worker replika di latar belakang. Karena proses pengiriman dan penerapan log bergantung pada latensi jaringan serta antrean lokal, perbedaan antara LSN primary dan applied LSN replika selalu muncul.

Async replication optimizes write latency by accepting eventual consistency on reads.

Garansi Read-Your-Own-Writes

User harus selalu melihat update yang baru saja dilakukannya sendiri

DEFINISI FORMAL

Diformalkan oleh Terry et al. (1994) sebagai jaminan sesi terdistribusi
Jika operasi W1 dilakukan oleh sesi S pada waktu t, maka semua read R berikutnya oleh sesi S pada t' > t wajib mengamati efek dari W1

SKOP GARANSI

Berlaku khusus untuk sesi atau user yang melakukan penulisan
User lain diperbolehkan melihat data yang sedikit tertinggal (stale)
Menghindari kebutuhan linearizability penuh pada seluruh cluster

STRATEGI PENYELESAIAN

Time-Based Sticky: Paksa read ke primary selama jendela waktu tertentu
Causal Token / Min LSN: Tunggu replika mengejar LSN commit transaksi
Synchronous Mode: Terapkan perubahan ke semua replika sebelum commit

Garansi read-your-own-writes memastikan pengguna selalu dapat membaca hasil perubahannya sendiri sesaat setelah commit berhasil. Berbeda dari konsistensi ketat global, aturan ini dibatasi pada level sesi pengguna. Pengguna lain di belahan sistem lain masih dapat menerima data eventual consistency, sehingga sistem tetap memiliki skalabilitas baca yang tinggi tanpa mengorbankan pengalaman pengguna penulis.

Read-your-own-writes is a session guarantee, not global linearizability.

Mengapa Naive Read-Split Gagal

Memisahkan write ke primary dan read round-robin ke replica memicu bug

```
func (r *Router) ReadNaive(key string) {
    target := replicas[rrIndex.Add(1) % len(replicas)]
    return target.Read(key)
}
```

Pola pemisahan read-write konvensional mengarahkan seluruh operasi tulis ke primary dan membagi query baca secara round-robin ke seluruh replika tanpa memeriksa status replikasi. Pada mesin pengembangan lokal dengan latensi nol, pola ini tampak berjalan sempurna. Namun di lingkungan produksi dengan beban I/O nyata, replika tertinggal beberapa ratus milidetik dan memicu anomali baca kosong.

URUTAN KEGAGALAN

1. Client Write('user:profile:123', 'Alice') -> Primary LSN 1
2. Client ReadNaive('user:profile:123') -> Replika-2 (Applied LSN 0)
3. Replika-2 mengembalikan ErrNotFound karena WAL belum di-apply

TITIK BUTA PENGEMBANG

Di lingkungan lokal, lag bernilai ~0ms sehingga bug tidak pernah muncul
Di staging, volume data rendah menyembunyikan antrean replikasi
Di produksi dengan network latency dan disk I/O, bug meledak sporadis

Naive round-robin blindly routes reads to replicas without freshness verification.

Arsitektur Lab 33: Tiga Layer Utama

Cluster simulator in-memory, Dynamic Router, dan Test Verifier

KOMPONEN CLUSTER

Node: Menyimpan map KV, appliedLSN, dan antrean channel WAL
Primary: Atomic increment currentLSN, eksekusi write real-time
Replica: Worker goroutine dengan delay lag terkonfigurasi dan sync.Cond

DYNAMIC ROUTER

Session State: Tracking LastWriteTime dan LastWriteLSN via sync.Map
Strategi Routing: Naive, Sticky Session, Causal Token, Lag-Aware
Primary Fallback: Pengalihan otomatis saat replika melebihi batas toleransi

VERIFIKASI SISTEM

Test Suite deterministik: 7 skenario pengujian unit dan integrasi
Race detector: Bebas data race pada ratusan goroutine konkuren

Arsitektur sistem Lab 33 dirancang memisahkan mekanisme replikasi fisik dari lapisan perutean query klien. Lapisan Cluster mensimulasikan penyimpanan data in-memory dengan nomor urut log LSN dan goroutine worker replika. Lapisan Router memelihara status sesi pengguna dan menjalankan strategi pemilihan simpul baca berdasarkan kebijakan kesegaran data yang diinginkan.

Layered separation decouples cluster replication mechanics from client routing decisions.

Primary Node & Monotonic LSN Tracking

Setiap operasi tulis menaikkan sequence number global secara atomic

```
type WALEntry struct {
    LSN       uint64
    Key       string
    Value     string
    Timestamp time.Time
}
```

```
lsn := c.currentLSN.Add(1)
c.primary.mu.Lock()
c.primary.data[key] = value
c.primary.appliedLSN = lsn
c.primary.mu.Unlock()
```

Untuk menetapkan urutan mutasi data yang tidak terbantahkan, simpul primary memanfaatkan pencacah atomik 64–bit yang merepresentasikan Log Sequence Number. Setiap kali transaksi tulis terjadi, primary memperbarui data lokal, mencatat nomor urut LSN terbaru, dan membungkus data tersebut ke dalam paket entri WAL untuk didistribusikan ke simpul replika.

KARAKTERISTIK LSN

Monotonik strictly increasing tanpa gap
Menjadi representasi posisi kemajuan commit log basis data
Dasar penentuan komparasi kesegaran data antar simpul

Monotonic LSN provides a global logical clock for ordering database state.

Replica Worker & Simulasi Lag Terkontrol

Worker goroutine menerapkan jeda waktu sebelum memperbarui memory dan LSN

```
for entry := range replica.walChannel {
    if lag > 0 { time.Sleep(lag) }
    replica.mu.Lock()
    replica.data[entry.Key] = entry.Value
    replica.appliedLSN = entry.LSN
    replica.cond.Broadcast()
    replica.mu.Unlock()
}
```

Setiap replika menjalankan sebuah goroutine worker yang membaca antrean channel WAL berkapasitas 1024 entri. Durasi lag disimulasikan melalui jeda waktu terkontrol sebelum data disinkronkan ke memori lokal. Begitu data diterapkan, variabel kondisi sync.Cond memicu sinyal siaran guna membangunkan operasi baca yang sedang menunggu pencapaian LSN tertentu.

MEKANISME NOTIFIKASI

sync.Cond digunakan untuk mem-broadcast event ke reader yang menunggu
Channel buffer 1024 mencegah blocking write pada primary
Lag dapat diubah secara dinamis melalui SetLag(d) saat pengujian

KONDISI REPLIKASI

Selama jeda lag berlangsung, AppliedLSN simpul tetap bernilai lama
Query yang masuk pada rentang ini akan membaca snapshot usang

Replicas lag when ingestion outpaces application speed or network delays intervene.

Strategi 1: Time-Based Sticky Routing

Arahkan read ke primary selama jendela waktu T_sticky setelah write

```
if stateVal, ok := r.sessions.Load(sessionID); ok {
    state := stateVal.(SessionState)
    if time.Since(state.LastWriteTime) < r.config.StickyDuration {
        return r.cluster.Primary().Read(key)
    }
}
return r.ReadLagAware(key)
```

Strategi sticky routing berbasis waktu mencatat stempel waktu penulisan terakhir dari masing-masing sesi pengguna. Setiap operasi baca yang dilakukan dalam rentang jendela waktu TTL akan dialihkan langsung ke primary. Pendekatan ini sangat efisien karena tidak membutuhkan pengecekan status simpul, meskipun bersifat heuristik dan bergantung pada estimasi latensi.

KEUNGGULAN OPERASIONAL

Implementasi sangat ringan: hanya membandingkan timestamp lokal
Tidak memerlukan query status LSN atau koordinasi antar simpul
Secara drastis mereduksi anomali post-write pada navigasi umum user

KELEMAHAN HEURISTIK

Jendela 5 detik adalah estimasi, bukan jaminan pasti replika sudah catch-up
Beban primary tetap tinggi jika user aktif menulis dan membaca bersamaan

Time-based sticky routing trades primary capacity for simplicity without cluster round-trips.

Strategi 2: Causal Token / Minimum LSN Routing

Client membawa commit token LSN dan menunggu replica mengejar posisi tersebut

```
for _, replica := range replicas {
    if replica.AppliedLSN() >= minLSN {
        return replica.Read(key)
    }
}
```

```
waitCtx, cancel := context.WithTimeout(ctx, r.config.WaitTimeout)
defer cancel()
if err := targetReplica.WaitForLSN(waitCtx, minLSN); err == nil {
    return targetReplica.Read(key)
}
return r.cluster.Primary().Read(key) // Fallback aman
```

Pendekatan causal token menyertakan token LSN hasil transaksi tulis pada setiap query baca berikutnya. Router memeriksa apakah terdapat replika yang telah mencapai nomor urut tersebut. Jika belum ada, sistem menunggu secara efisien menggunakan variabel kondisi hingga batas waktu tercapai, lalu melakukan fallback ke primary bila replika mengalami keterlambatan parah.

GARANSI MATEMATIS

Pasti mengamati mutasi transaksi karena AppliedLSN >= minLSN terjamin
Jika replika lambat, fallback ke primary melindungi ketersediaan sistem

Causal token routing guarantees causal consistency by conditioning reads on commit LSN.

Event-Driven WaitForLSN dengan sync.Cond

Menghilangkan polling CPU yang boros dengan sinkronisasi variabel kondisi

```
func (n *Node) WaitForLSN(ctx context.Context, target uint64) error {
    go func() {
        n.mu.Lock(); defer n.mu.Unlock()
        for n.appliedLSN < target {
            n.cond.Wait() // Dibangunkan oleh Broadcast worker
        }
        close(ch)
    }()
    select {
    case <-ctx.Done(): return ctx.Err()
    case <-ch: return nil
    }
}
```

Alih-alih membebani CPU dengan mekanisme polling berulang, method WaitForLSN menidurkan goroutine pembaca menggunakan sync.Cond. Ketika worker replika selesai menerapkan transaksi WAL, sinyal siaran dikirimkan untuk membangunkan goroutine secara instan, menghasilkan waktu tunggu yang presisi dan hemat sumber daya.

EFISIENSI KOMPUTASI

Goroutine reader tertidur tanpa memakan siklus komputasi CPU
Langsung aktif seketika worker replika selesai menerapkan WAL
Mendukung pembatalan timeout context secara aman dan bersih

SAFETY CONCURRENCY

Mutex melindungi pembacaan dan penulisan variabel appliedLSN

Event-driven condition variables eliminate busy-waiting during replication catch-up.

Strategi 3: Lag-Aware Dynamic Routing

Mengecualikan replica yang tertinggal jauh dan fallback saat replica habis

```
primaryLSN := r.cluster.CurrentLSN()
for _, replica := range replicas {
    diff := primaryLSN - replica.AppliedLSN()
    if diff <= r.config.MaxLSNDiff {
        eligible = append(eligible, replica)
    }
}
```

```
if len(eligible) == 0 {
    return r.cluster.Primary().Read(key) // Fallback-lag
}
idx := r.rrIndex.Add(1) % len(eligible)
return eligible[idx].Read(key)
```

Untuk operasi baca umum yang tidak terikat sesi penulis, strategi lag-aware menghitung selisih delta antara LSN primary dan LSN terapan replika. Simpul yang melewati batas toleransi MaxLSNDiff dikeluarkan dari antrean beban. Jika seluruh replika mengalami gangguan, router secara otomatis mengalihkan beban baca ke primary demi menjaga integritas data.

PROTEKSI SISTEM

Mencegah query umum membaca data yang terlalu usang
Mengisolasi simpul replika yang mengalami gangguan I/O atau jaringan
Degradasi anggun (graceful degradation) menjaga ketersediaan layanan

Dynamic lag-aware routing preserves consistency SLAs by shedding severely lagging nodes.

Strategi 4: Synchronous Replication (remote_apply)

Write diblokir hingga semua replica menerapkan perubahan secara tuntas

```
if c.replMode == SyncReplication {
    for _, replica := range c.replicas {
        if lag > 0 { time.Sleep(lag) }
        replica.data[key] = value
        replica.appliedLSN = lsn
    }
}
```

Replikasi sinkron menahan konfirmasi transaksi tulis pada primary hingga seluruh simpul replika selesai menerapkan perubahan ke penyimpanan lokal. Pola ini sepenuhnya melenyapkan anomali stale read pada semua replika, namun mengorbankan performa tulis secara drastis karena setiap operasi penulisan terhambat oleh latensi jaringan terjauh.

GARANSI KESEGARAN

Semua replika seketika memiliki AppliedLSN == PrimaryLSN
Naive read sekalipun dijamin 100% bebas dari anomali stale read
Setara dengan parameter synchronous_commit = remote_apply di PostgreSQL

BIAYA PERFORMA TULIS

Latensi write membengkak sebanding dengan round-trip terlama replika
Satu replika lambat akan memperlambat seluruh proses penulisan sistem
Hanya direkomendasikan untuk sistem perbankan atau data finansial kritis

Synchronous replication achieves immediate read freshness by penalizing write latency.

Perbandingan 4 Strategi Mitigasi Lag

Evaluasi matriks trade-off antara garansi, latensi, dan beban primary

Strategi	Garansi	Latensi Read	Overhead Primary	Kompleksitas
Naive Read	Eventual	Terendah	Nol	Sangat Rendah
Sticky Session	Read-Own-Writes	Terendah	Sedang (T_sticky)	Rendah
Causal Token	Causal Session	Sedang (Wait)	Rendah (Fallback)	Sedang
Lag-Aware	Bounded Lag	Rendah	Kondisional	Sedang
Sync Mode	Linearizable	Terendah	Nol (Write Mahal)	Tinggi

Pemilihan strategi penanganan replication lag menuntut keseimbangan antara garansi konsistensi, latensi query, dan beban kerja simpul primary. Sticky routing menawarkan implementasi paling praktis untuk alur pengguna umum, sedangkan causal token memberikan kepastian matematis tanpa membebani primary secara berlebihan.

ANALISIS KEBUTUHAN

E-commerce umum: Kombinasi Sticky Session (checkout) + Lag-Aware (katalog)
Fintech / Ledger: Causal Token untuk transaksi atau Synchronous Replication
Analytics Dashboard: Lag-Aware dengan toleransi batas LSN yang longgar

Match your replication mitigation strategy to the cost of reading stale state.

Tidak ada solusi perak: pilih strategi berdasarkan nilai toleransi bisnis

Hasil Eksekusi Demo CLI Lab 33

Bukti empiris terjadinya anomali dan keberhasilan masing-masing strategi

```
[Write Primary] key='user:profile:123', LSN: 1
[Naive Read] Node: replica-2, Applied LSN: 0, Err: key not found
[Sticky Read (within 500ms)] Node: primary, LSN: 1, Val: Alice
[Sticky Read (after TTL)] Node: replica-1, LSN: 1, Val: Alice
```

```
[Write Primary] key='order:101', LSN: 2
[Token Read (MinLSN=2)] Node: replica-1, LSN: 2, Val: completed
[Sync Write] LSN: 1, Write Duration: 102.19ms
[Sync Naive Read] Node: replica-2, LSN: 1, Val: Bob, Err:
```

Eksekusi demo CLI membuktikan validitas seluruh skenario secara nyata. Pendekatan naif langsung memicu error data tidak ditemukan saat lag aktif, sticky routing berhasil melindungi pembacaan pengguna ke primary, causal token menunggu hingga replika siap secara presisi, dan mode sinkron mencatatkan penalti waktu tulis terukur sebesar 102ms.

INTERPRETASI HASIL

Skenario 1 membuktikan kegagalan fatal metode naif saat lag aktif
Skenario 2 membuktikan transisi mulus dari primary ke replika setelah TTL
Skenario 3 membuktikan penungguan deterministik hingga replika siap
Skenario 4 membuktikan kesegaran instan dengan penalti durasi tulis 102ms

Empirical demo execution validates theoretical invariants under controlled conditions.

Verifikasi 7 Test Case & Bebas Data Race

Pengujian unit dan konkurensi membuktikan ketahanan sistem di bawah beban

```
TestNaiveReplicationLag_StaleRead: PASS (0.60s)
TestStickySessionRouting: PASS (0.55s)
TestReadWithToken_LSN: PASS (0.20s)
TestReplicaLagThreshold_Fallback: PASS (0.00s)
TestSynchronousReplication_Freshness: PASS (0.10s)
TestConcurrentAccess_RaceFree: PASS (1.10s)
TestWaitForLSN_ContextTimeout: PASS (0.05s)
```

Seluruh 7 skenario pengujian unit lulus sempurna dengan pengaktifan Go race detector. Suite pengujian memvalidasi munculnya anomali stale read, transisi TTL sticky session, penungguan token LSN, fallback akibat lag berlebih, kesegaran replikasi sinkron, serta keamanan akses memori konkuren tanpa data race.

UJI KONKURENSI EKSTRIM

5 Goroutine Writer + 10 Goroutine Reader
Masing-masing 20 operasi paralel (Total 300 operasi simultan)
Dijalankan dengan go test -race tanpa memicu race condition

UJI BATAS WAKTU

TestWaitForLSN membuktikan pengembalian context.DeadlineExceeded
Sistem tidak mengalami kebocoran memori atau dead lock goroutine

Automated test assertions with race detection guarantee thread-safe correctness.

Pemetaan ke PostgreSQL dan MySQL

Bagaimana konsep lab berkorespondensi dengan konfigurasi database populer

POSTGRESQL PRIMITIVES

LSN: pg_current_wal_lsn() & pg_last_wal_replay_lsn()
Sync Commit: synchronous_commit = on | remote_apply | off
Lag Metric: pg_stat_replication.replay_lag
Catch-Up Query: pg_wal_replay_pause() / pg_last_wal_receive_lsn()

MYSQL / VITESS PRIMITIVES

GTID: Global Transaction Identifier setara dengan LSN monotonik
Semi-Sync: rpl_semi_sync_master_enabled (durability, bukan visibility)
Lag Metric: Seconds_Behind_Master (kurang akurat) vs Heartbeat GTID
WAIT_FOR_EXECUTED_GTID_SET: Ekivalen native dari WaitForLSN

PERINGATAN SEMI-SYNC

MySQL semi-sync hanya menjamin relay log sampai di replika
Data belum tentu sudah di-apply ke storage engine (bisa tetap stale)

Konsep yang disimulasikan pada Lab 33 memiliki padanan langsung pada basis data produksi. PostgreSQL menyediakan fungsi pemantauan LSN fisik dan konfigurasi synchronous_commit, sedangkan MySQL mengandalkan Global Transaction Identifiers (GTID) serta fungsi WAIT_FOR_EXECUTED_GTID_SET untuk sinkronisasi kausal deterministik.

Database engines provide native LSN/GTID primitives; applications orchestrate routing.

Replication Lag di Cloud Managed Services

Perbedaan karakteristik arsitektur replikasi shared-storage vs shared-nothing

AWS AURORA (SHARED STORAGE)

Arsitektur storage terdistribusi: Primary dan replica membaca volume sama
Replication lag sangat rendah (umumnya < 20ms)
Lag hanya berupa invalidasi buffer cache memori lokal di replica

AWS RDS / AZURE PG (SHARED NOTHING)

Replikasi streaming berbasis pengiriman byte WAL fisik antar instans
Lag berfluktuasi drastis saat terjadi transaksi batch besar atau spike I/O
Metrik: ReplicaLag (CloudWatch) & physical_replication_delay (Azure)

IMPLIKASI DESAIN

Aurora tetap membutuhkan mitigasi read-your-own-writes pada microsecond
Shared-nothing membutuhkan threshold MaxLSNDiff dan fallback yang ketat

Layanan cloud terkelola memiliki arsitektur replikasi yang berbeda. AWS Aurora memisahkan komputasi dari penyimpanan terdistribusi bersama, sehingga replika hanya mengalami lag pembaruan cache memori dalam orde milidetik rendah. Sebaliknya, RDS standar dan Azure PG menggunakan aliran WAL konvensional yang rentan terhadap lonjakan lag saat lalu lintas data padat.

Shared storage reduces replication lag magnitude but does not eliminate cache staleness.

Tantangan Scaling Session State di Produksi

Transisi dari sync.Map single-process menuju arsitektur stateless

BATASAN IN-MEMORY MAP

Router lab menyimpan sesi pada sync.Map lokal di satu instance aplikasi
Pada arsitektur horizontal (banyak pod API), map lokal tidak tersinkronisasi
Request berikutnya dari user bisa masuk ke pod yang tidak memiliki state sesi

SOLUSI 1: DISTRIBUTED CACHE

Simpan LastWriteTime dan LastWriteLSN di Redis dengan TTL terkonfigurasi
Semua pod API membaca status sesi dari cluster Redis terpusat
Biaya: Tambahan satu round-trip network lookup sebelum eksekusi query

SOLUSI 2: CLIENT TOKEN (JWT/COOKIE)

Kirim commit LSN kembali ke client via header HTTP response
Client mengirimkan kembali LSN via cookie/header pada request berikutnya
Stateless penuh pada backend, router langsung membaca nilai token dari header

Implementasi lab menggunakan sync.Map in-memory untuk melacak status sesi pada satu proses aplikasi. Pada sistem produksi berskala horizontal dengan banyak container, status sesi harus didistribusikan. Pendekatan praktis meliputi penyimpanan stempel sesi di Redis atau menyematkan token LSN secara stateless ke dalam cookie terenkripsi milik klien.

Stateless routing pushes causal tokens to client cookies or shared Redis clusters.

Overhead Polling LSN & Optimasi Heartbeat

Menghindari query metadata basis data berulang pada setiap request pembacaan

MASALAH PER-QUERY LSN CHECK

Menjalankan query LSN ke database pada setiap read melipatgandakan round-trip
Beban CPU basis data tersita untuk melayani query metadata, bukan query data
Koneksi pool cepat jenuh akibat query administratif yang berulang

SOLUSI BACKGROUND HEARTBEAT

Worker aplikasi melakukan polling lag replica secara berkala (misal tiap 100ms)
Status LSN dan kelaikan simpul dicatat di memory cache lokal aplikasi
Router membuat keputusan perutean murni berdasarkan data cache tanpa I/O

HEARTBEAT TABLE TECHNIQUE

Primary menulis timestamp periodik ke tabel khusus __heartbeat
Replika mereplikasi baris tersebut, router membaca selisih waktu secara pasif

Mengeksekusi query pemeriksaan LSN ke replika pada setiap request baca dapat memicu overhead round-trip yang masif. Arsitektur berkinerja tinggi mengatasi kendala ini dengan menjalankan worker background asinkron yang memperbarui metrik lag secara periodik ke dalam memori aplikasi, sehingga keputusan routing dapat diambil tanpa latensi tambahan.

Cache replica lag status asynchronously via heartbeats to avoid per-query metadata overhead.

Checklist Penerapan Read Replica di Produksi

Panduan verifikasi arsitektur sebelum mengaktifkan perutean replika

CHECKLIST ARSITEKTUR

[ ] Identifikasi query kritis pasca-write dan pasang garansi sesi
[ ] Konfigurasi threshold MaxLSNDiff dan aktifkan fallback primary otomatis
[ ] Pasang monitoring metrik lag replika pada persentil p95 dan p99
[ ] Pastikan ukuran connection pool replika memadai untuk lonjakan read
[ ] Tetapkan batas waktu timeout yang ketat saat menunggu replika catch-up
[ ] Uji skenario failover dan isolasi replika yang mengalami disk stall

ATURAN THRESHOLD

Gunakan metrik lag berbasis byte/LSN, bukan hanya unit detik
Alarm harus berbunyi sebelum antrean replika menyebabkan OOM pada worker

DISIPLIN REKAYASA

Selalu siapkan jalur fallback ke primary untuk melindungi ketersediaan

Sebelum mendistribusikan lalu lintas baca ke replika sekunder di lingkungan produksi, tim pengembang wajib menerapkan parameter operasional yang ketat. Siapkan mekanisme fallback otomatis ke primary, pantau metrik lag replikasi pada persentil p99, serta pastikan alur query pasca-tulis dilindungi oleh garansi sesi read-your-own-writes.

Enforce strict lag SLAs and automated primary fallbacks before scaling reads.

Batasan Model & Ruang Lingkup Lab 33

Transparansi batasan simulasi in-memory dibanding database fisik

YANG DIMODELKAN (TERSEDIA)

Aliran data WAL terurut dengan LSN monotonik atomik
Simulasi lag deterministik via channel buffer dan time delay
Empat strategi routing lengkap dengan fallback dan event synchronization
Pengujian konkurensi multithreading bebas race condition

BATASAN LAB (BELUM DIMODELKAN)

Penyimpanan murni in-memory map tanpa persistensi disk / fsync
Tidak memodelkan engine SQL, indexing B-Tree, atau parsing transaksi
Tidak memodelkan topologi cascading replication atau partisi multi-region
Tidak memodelkan algoritma konsensus pemilihan primary (Raft / Paxos)

TUJUAN EDUKASI

Membuktikan logika mitigasi lag tanpa kompleksitas instalasi DB eksternal

Lab 33 berfokus murni pada pembuktian logika perutean query dan garansi konsistensi sesi. Dengan membangun kluster key-value in-memory yang dilengkapi pencatatan LSN atomik, sistem mampu mendemonstrasikan anomali lag dan solusinya secara deterministik tanpa hambatan instalasi basis data eksternal yang rumit.

Simulations prove architectural invariants cleanly by stripping away disk and network noise.

Prinsip Inti Mengelola Replication Lag

Ringkasan pemikiran arsitektural untuk sistem basis data terdistribusi

1. ASINKRON ADALAH DEFAULT

Semua database populer berjalan asinkron demi mempertahankan throughput
Mengharapkan konsistensi instan dari replika tanpa strategi adalah ilusi

2. GARANSI BERBASIS KEBUTUHAN

Gunakan Sticky Routing untuk alur pengguna umum yang berbiaya murah
Gunakan Causal Token saat kebenaran matematis mutlak dibutuhkan

3. DEGRADASI ANGGUN

Batasi replika dengan MaxLSNDiff dan selalu sediakan fallback ke primary
Sistem yang baik tetap melayani pembacaan meski seluruh replika tertinggal

Replikasi asinkron tetap menjadi pilihan standar industri untuk mencapai throughput penulisan yang tinggi. Menjaga integritas data pengguna pasca-transaksi menuntut kecerdasan pada lapisan perutean query. Kombinasi antara sticky routing yang ringan dan causal token yang presisi menghasilkan sistem yang cepat sekaligus konsisten.

Asynchronous replication trades consistency for throughput; route by session when freshness matters.

Eksperimen kecil, interpretasi yang teliti.

Replication lag is inevitable in async systems; smart routing bridges the gap to consistency.

Software Engineering Lab 33 - Read Replicas & Replication Lag

internal/cluster/cluster.go (Node, WAL, Replication)
internal/router/router.go (Sticky, Token, Lag-Aware)
tests/replication_test.go (7 Test Cases, Race Free)
cmd/demo/main.go (CLI Interactive Demonstration)

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/33-read-replicas-and-replication-lag`

Lukman / lukman-ss

Penulis: Lukman / lukman-ss
Commit Hash: cfba3b76824f0ee03b5fa526462d1b72858144e7
Status: PASS (go test -v -race ./...)
Model: In-Memory Primary-Replica Simulation